

LAB 2 · 순차논리 연습

전자전기컴퓨터설계실험II

클록·저장·상태 전이를 파형으로 설명한다.

9월 21일 실습을 위한 공통 안내

작성일 2026. 09. 10. · 이해리

목차

- 01 · 이번 주 실험과 프로젝트 시작
 - 02 · 클록·리셋·이전 값·버튼 입력
 - 03 · 8개 회로에서 반드시 확인할 것
 - 04 · 시뮬레이션·수정 실험·실험 전후 레포트
- 시작 공통 개념 회로별 검사 레포트

이번 주 필수 실험 · 1-4

교육 번호	실험
11	업/다운 카운터
12	클록 분주
13	레지스터: 데이터 저장과 이동
14	시프트 레지스터

8개 모두 예습한다. 실습에서는 지정된 회로를 설명하고 시연한다.

이번 주 필수 실험 · 5-8

교육 번호	실험
15	PISO
16	Moore 상태 머신
17	Mealy 상태 머신
18	7세그먼트 자리 스캔

8개 모두 예습한다. 실습에서는 지정된 회로를 설명하고 시연한다.

한 회로를 끝내는 순서

1. 태그 고정 템플릿 clone → 새 VS Code 창에서 workspace 열기.
2. RTL·TB·설정을 직접 입력 → 저장 → VS Code 시뮬레이션.
3. 정상·변경·복구 로그와 파형을 실험 전 레포트에 정리.
4. Vivado GUI에서 RTL·TB·XDC 등록 → 시뮬레이션 → bit 생성.
5. 실제 보드 관찰·사진·영상·GitHub 링크를 실험 후 레포트에 정리.

같은 템플릿, 다른 폴더 이름

Windows PowerShell에서 한 줄씩 실행한다. 줄 끝의 ‘는 이어 쓰기다.

```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_01_counter
cd lab2_01_counter
git switch -c main
code LAB1.code-workspace
```

다음 실험은 마지막 폴더 이름만 바꾼다. LAB1.code-workspace는 공통 템플릿 파일명이므로 그대로 연다.

VS Code 새 창에서 시작

1. File → New Window를 누른다. 단축키는 Ctrl+Shift+N.
2. File → Open Workspace from File...을 누른다.
3. clone한 폴더에서 LAB1.code-workspace를 선택하고 Open.
4. Explorer의 PROJECT 아래 src·sim·constraints·tools를 확인한다.
5. Extensions에서 @recommended를 검색해 추천 확장을 설치한다.

직접 작성할 파일

위치	역할
src/	FPGA에 구현할 모듈
sim/	입력 자극·기대값·종료 조건을 가진 TB
constraints/	장비 핀과 전기 규격을 지정할 XDC
simulation.json	RTL 경로·TB 경로·TB 모듈명

빈 파일에 교안의 전체 코드를 입력한다. 파일명과 module 이름은 서로 다른 항목이다.

시간을 읽는 기준

이번 TB의 클록 주기는 10ns다. 상승 에지는 5, 15, 25, ... ns에 온다.

입력을 바꿔도 레지스터는 다음 상승 에지에서 갱신된다.

TB는 상승 에지 뒤 1ns 기다린 다음 결과를 비교한다. nonblocking 갱신 전의 값을 잘못 검사하지 않기 위해서다.

시뮬레이션의 10ns와 실제 장비의 클록 설정은 별개다.

동기 리셋과 enable

조건	다음 상승 에지의 동작
$\text{rst} = 1$	초기값으로 변경
$\text{rst} = 0, \text{enable} = 1$	회로가 정한 상태 갱신
$\text{rst} = 0, \text{enable} = 0$	이전 상태 유지

리셋을 올린 순간과 그 다음 상승 에지를 파형에서 구분한다.

각 모듈의 리셋 우선순위를 읽는다. Mealy의 조합 출력은 입력에도 의존한다.

nonblocking은 이전 값을 전달한다

두 레지스터 A와 B가 있고, 같은 에지에서 A에 입력을 저장하고 B에 A를 전달한다고 하자.

에지 직전	입력	에지 직후
A = 0xA, B = 0	3	A = 3, B = 0xA
A = 3, B = 0xA	3	A = 3, B = 3

0xA는 16진수 표기다. B는 같은 에지에서 갱신되기 전 A를 받는다.

순차회로의 저장값은 <=로 갱신한다. 조합 계산은 별도 always @*에서 =를 사용한다.

버튼을 클록으로 쓰기 전에

기계식 버튼은 한 번 눌러도 접점이 여러 번 튕 수 있다. FPGA 클록과도 동기화되어 있지 않다.

최신판의 입력 처리 순서는 동기화 → 안정 시간 확인 → 한 클록 펄스다.

동기화 플립플롭 두 단계는 메타안정성이 전달될 위험을 낮춘다.
디바운스는 접점의 반복 변화를 걸러낸다.

시뮬레이션은 메타안정성의 아날로그 현상 자체를 검증하지 않는다. 버튼 잡음 자극과 펄스 수를 검사한다.

clock-enable과 분주 출력

분주 출력은 느린 주기를 LED나 파형으로 관찰할 때 사용한다.

다른 회로의 저장소는 공통 clk를 유지하고, tick이 1인 상승 에지에서만 동작시킨다.

$\text{DIVISOR} = 10$ 이면 tick이 열 클록마다 한 번 소비된다. tick은 소비할 에지 직전부터 1이다.

실제 클록 주파수와 분주 수의 비가 원하는 속도인지 계산한다.

11 · 업/다운 카운터

자극	기대 결과
증가 16회	$0 \rightarrow \dots \rightarrow 15 \rightarrow 0$
0에서 감소	15
enable = 0	값 유지
rst와 enable 모두 1	리셋 우선

수정 실험: 증가량을 1에서 2로 바꾼다. 첫 증가 에지의 기대값과 실제값을 비교한 뒤 복구한다.

12 · 클록 분주

DIVISOR = 10	관찰
리셋 해제 뒤 1~4번째 에지	출력 0
5번째 에지	출력 1
10번째 에지	출력 0, 주기 반복
30클록	enable 소비 3회

수정 실험: 첫 전이를 한 클록 늦춘다. 주기가 같더라도 듀티와 첫 전이가 달라졌는지 확인한다.

13 · 레지스터 저장과 이동

제어	기대 결과
load만 1	stored에 입력 저장
transfer만 1	value에 stored 전달
둘 다 1	value는 이전 stored를 받음
둘 다 0	두 저장값 유지

수정 실험: 전달 대상을 stored에서 현재 data_in으로 바꾼다. 입력과 저장값이 다른 경우를 관찰한다.

14 · 시프트 레지스터

순서대로 넣는 입력	저장값
1	1000
0	0100
1	1010
0	0101

새 입력은 bit 3으로 들어오고 기존 값은 bit 0 방향으로 이동한다.

수정 실험: 이동 방향을 반대로 바꾸고 첫 불일치 위치를 찾는다.

15 · PISO

1010을 병렬 로드하면 직렬 출력은 먼저 최상위 비트 1이다.

읽는 시점	직렬 출력
로드 뒤, 첫 shift 전	1
한 번 shift 뒤	0
두 번 shift 뒤	1
세 번 shift 뒤	0

네 번째 shift 뒤 저장값은 0000이다. load와 enable이 동시에 1이면 load가 우선한다.

16 · Moore 상태 머신

현재 상태	advance = 1인 유효 에지 뒤
00	01
01	10
10	00

enable과 advance가 모두 1일 때만 전이한다.

출력은 상태다. 클록 사이에 advance만 바뀌어도 출력은 바뀌지 않는다.

수정 실험: 01의 다음 상태를 00으로 바꾸고 원래 전이표와 비교한다.

17 · Mealy 상태 머신

상태	입력 0 출력	입력 1 출력
S0	00	10
S1	00	01

유효 에지에서 입력이 1이면 S0↔S1로 전이한다. 입력이 0이면 유지한다.

출력은 현재 상태와 현재 입력에 의존한다. 입력 1을 유지한 채 전이하면 에지 뒤 출력도 바뀐다.

enable = 0은 상태만 멈춘다. 입력에 따른 조합 출력 변화는 계속된다.

18 · 7세그먼트 자리 스캔

8자리 중 한 자리만 켜고 순서대로 반복한다. 눈에는 여러 자리가 함께 보인다.

자리 전환 사이에는 모든 자리 선택을 0으로 하는 blank 구간을 둔다.
다음 자리 데이터가 안정될 시간을 확보한다.

기능 코어는 active-high 논리값을 낸다. 실제 보드의 선택·세그먼트 극성은 핀 연결 모듈에서 맞춘다.

수정 실험: blank 구간을 제거하고 자기검사가 어떤 조건에서 실패하는지 찾는다.

VS Code 시뮬레이션 실행

1. File → Save All로 RTL·TB·설정을 저장한다.
2. Terminal → Run Task... → 02 Simulate를 누른다.
3. Terminal에서 LAB2_PASS와 검사 수, 종료 시각을 확인한다.
4. Terminal → Run Task... → 03 Open waveform을 누른다.
5. clk·rst·제어 입력·상태·출력을 추가하고 상승 에지를 확대한다.

오류를 읽는 순서

1. 문법 오류: 파일 경로와 행 번호를 확인한다. 이전 행의 세미콜론도 확인한다.
2. 자기검사 실패: LAB2_FAIL 뒤 검사 이름과 시각을 읽는다.
3. 그 시각 직전의 입력과 현재 상태로 기대값을 다시 계산한다.
4. RTL만 수정하고 TB 기대값을 임의로 바꾸지 않는다.
5. 복구·저장·재실행 후 새 PASS 로그와 새 파형을 보관한다.

실험 전 레포트

1. 회로의 상태표·리셋·enable·출력 시점을 설명한다.
2. 주요 RTL과 TB의 역할을 자기 말로 설명한다.
3. 정상/변경/복구의 3행 비교표에 로그와 결과를 정리한다.
4. 상승 에지 전후를 확대한 VS Code 파형으로 예상값을 설명한다.
5. Vivado에 등록할 RTL·TB·XDC·top·장비 클록 조건을 적는다.

실험 후 레포트

1. Vivado 기능 시뮬레이션과 VS Code 결과를 비교한다.
2. 합성·구현 결과, 경고 해석, 핀과 타이밍 조건, bit 파일을 기록한다.
3. 장치 프로그램 화면과 실제 입출력 사진·영상을 첨부한다.
4. 통합판은 버튼 모드 전환·LCD 모드명·회로 출력을 함께 보여 준다.
5. GitHub에 소스·레포트·관찰 자료를 연결하고 미수행 항목을 구분한다.

v2.0.1 도구 설정 확인

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src·sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내